ONEDRAFT

CAD-AI — Aria Powered

API & Domain Schema Specification

Version 1.0

Status: Implementation Contract
Architecture: Model-Centric / Versioned / Service-Oriented
Primary Model: OneDraft Project Model
AI Layer: Aria Powered
Primary Document Standard: ARCH D
API Style: REST initially, event-driven internally
Identifier Standard: UUID-based immutable identifiers

---

1. PURPOSE

This document defines the canonical domain objects and application programming interfaces required to implement OneDraft.

The specification establishes:

what objects exist

how objects relate

what operations are permitted

which services own those operations

how model changes are versioned

how Aria interacts with the model

how drawings derive from the model

how regulatory controls attach to projects

how redlines become structured modifications

how final acceptance is recorded

The Project Model remains the single source of truth.

---

2. UNIVERSAL OBJECT STRUCTURE

All persistent OneDraft domain objects shall use a common identity structure.

Conceptually:

```
id
project_id
created_at
updated_at
created_by
updated_by
created_revision
modified_revision
status
metadata
```

Identifiers shall be UUIDs.

Human-readable identifiers may additionally exist.

For example:

```
id: 8f6...
element_number: WALL-00427
```

The UUID is the immutable machine identity.

The human-readable identifier is the user-facing identity.

---

3. PROJECT

Object

`Project`

Required Fields

```
id
organization_id
name
project_type
status
description
address
jurisdiction
municipality
province_state
country
latitude
longitude
```

```
unit_system
currency
current_revision_id
current_model_version
current_code_manifest_id
created_at
updated_at
```

Status

```
INTAKE
ACTIVE
REVIEW
REDLINE
FINAL_REVIEW
ACCEPTED
ARCHIVED
SUSPENDED
```

Relationships

A Project contains:

```
1..N Requirements
1 Site
1..N Buildings
1..N Revisions
1..N Code Manifests
1..N Drawings
1..N Redlines
1..N Validation Results
1..N Acceptance Records
1..N Events
```

---

4. ORGANIZATION

Object

`Organization`

Represents a customer tenant.

Fields:

```
id
name
organization_type
billing_account_id
status
created_at
updated_at
```

An organization may contain multiple users and projects.

---

5. USER

Object

`User`

Fields:

```
id
organization_id
email
display_name
role
status
created_at
updated_at
```

Roles initially include:

```
OWNER
ADMIN
DESIGNER
DRAFTER
ARCHITECT
ENGINEER
DEVELOPER
REVIEWER
CLIENT
```

Permissions shall be determined independently from display roles.

---

6. PROJECT REQUIREMENT

Object

`Requirement`

Fields:

```
id
project_id
source
description
requirement_type
priority
mandatory
status
```

```
verification_status
affected_objects
created_revision
modified_revision
```

Requirement types include:

```
FUNCTIONAL
SPATIAL
AESTHETIC
MATERIAL
ACCESSIBILITY
PERFORMANCE
BUDGET
SCHEDULE
SITE
REGULATORY
CUSTOM
```

A requirement may reference multiple project objects.

---

7. SITE

Object

`Site`

Fields:

```
id
project_id
boundary_geometry
north_angle
elevation_reference
topography
existing_conditions
site_constraints
setbacks
easements
rights_of_way
utilities
parking
access
survey_reference
```

The Site is a first-class geometric object.

---

8. BUILDING

Object

`Building`

Fields:

```
id
project_id
name
building_type
occupancy
construction_type
height
area
number_of_levels
geometry
status
```

A project may contain multiple buildings.

---

9. LEVEL

Object

`Level`

Fields:

```
id
building_id
name
level_code
elevation
floor_to_floor_height
finished_floor_elevation
ceiling_elevation
sequence
status
```

Example:

```
P2
P1
G
L1
L2
L3
ROOF
TRUSS
```

The `sequence` determines vertical ordering.

---

10. GRID

Object

`Grid`

Fields:

```
id
building_id
grid_type
label
geometry
sequence
```

Grid types:

```
ORTHOGONAL
RADIAL
CUSTOM
```

Grid objects provide the reference framework for structural and architectural geometry.

---

11. SPACE

Object

`Space`

Fields:

```
id
level_id
space_number
name
space_type
occupancy
area
volume
boundary_geometry
ceiling_height
accessibility_class
fire_classification
status
```

Examples:

```
BEDROOM
KITCHEN
BATHROOM
CORRIDOR
STAIR
MECHANICAL
ELECTRICAL
LOBBY
OFFICE
RETAIL
ASSEMBLY
```

---

12. ELEMENT

Object

`Element`

The Element is the universal semantic architectural object.

Fields:

```
id
project_id
level_id
parent_id
element_type
name
geometry_reference
assembly_id
parameters
host_element_id
status
```

Element types include:

```
WALL
DOOR
WINDOW
FLOOR
ROOF
SLAB
STAIR
RAMP
COLUMN
BEAM
OPENING
CEILING
PARTITION
CURTAIN_WALL
GUARD
HANDRAIL
FIXTURE
EQUIPMENT
```

```
MILLWORK
CUSTOM
```

---

13. ELEMENT HIERARCHY

Elements may contain child elements.

Example:

```
ROOF-001
  ├── ROOF-SLOPE-001
  ├── ROOF-ASSEMBLY-001
  ├── ROOF-OPENING-001
  └── ROOF-DRAIN-001
```

This permits complex assemblies without flattening the model.

---

14. ASSEMBLY

Object

`Assembly`

Fields:

```
id
project_id
name
assembly_type
layers
total_thickness
performance_properties
fire_rating
thermal_properties
acoustic_properties
manufacturer_data
```

An assembly may be referenced by many elements.

---

15. PARAMETER

Parameters shall not be hard-coded into individual services.

A parameter has:

```
name
```

```
value
unit
data_type
source
constraint
```

Examples:

```
wall_height = 3000 mm
wall_thickness = 200 mm
door_width = 900 mm
ceiling_height = 2400 mm
```

---

16. CONSTRAINT

Object

`Constraint`

Fields:

```
id
project_id
constraint_type
source
priority
value
unit
tolerance
affected_objects
status
```

Constraint types:

```
DIMENSION
ALIGNMENT
OFFSET
PARALLEL
PERPENDICULAR
CONCENTRIC
EQUAL
MINIMUM
MAXIMUM
CLEARANCE
ADJACENCY
CONTAINMENT
ACCESSIBILITY
CODE
STRUCTURAL
CUSTOMER
```

---

17. RELATIONSHIP

Object

`Relationship`

Relationships are explicit model objects where necessary.

Fields:

```
id
project_id
source_object_id
target_object_id
relationship_type
metadata
```

Relationship types include:

```
HOSTS
BOUNDS
ADJOINS
CONTAINS
SUPPORTS
REFERENCES
ALIGNS_WITH
DEPENDS_ON
CONNECTED_TO
LOCATED_ON
GENERATES
```

This relationship graph becomes the foundation of dependency propagation.

---

18. GEOMETRY REFERENCE

Geometry shall be separated from semantic identity.

Object

`GeometryReference`

Fields:

```
id
object_id
geometry_type
coordinate_system
geometry_version
storage_reference
bounding_box
geometry_hash
```

Possible geometry types:

```
POINT
LINE
CURVE
SURFACE
SOLID
MESH
COMPOSITE
```

---

19. MODEL VERSION

Object

`ModelVersion`

Fields:

```
id
project_id
revision_number
parent_version_id
created_by
created_at
state_hash
status
```

Every meaningful project modification creates a new model version.

---

20. REVISION

Object

`Revision`

Fields:

```
id
project_id
revision_number
parent_revision_id
reason
created_by
created_at
status
model_version_id
code_manifest_id
validation_summary
```

Statuses:

```
DRAFT
IN_PROGRESS
UNDER_REVIEW
REDLINE
VALIDATED
ACCEPTED
SUPERSEDED
```

---

21. CHANGE RECORD

Object

`ChangeRecord`

Fields:

```
id
revision_id
object_id
operation
before_state
after_state
reason
source
```

Operations:

```
CREATE
MODIFY
MOVE
ROTATE
RESIZE
DELETE
REPLACE
REGENERATE
```

The ChangeRecord allows the system to reconstruct what happened during a revision.

---

22. ARIA COMMAND

Object

`AriaCommand`

This is the principal interface between AI reasoning and the Project Model.

Fields:

```
id
project_id
revision_id
operation
target_objects
parameters
preserve_constraints
reason
confidence
requested_by
validation_requirements
status
```

Statuses:

```
PROPOSED
VALIDATING
APPROVED
EXECUTING
COMPLETED
REJECTED
REQUIRES_REVIEW
FAILED
```

---

23. PERMITTED ARIA OPERATIONS

Initial operations:

```
CREATE_PROJECT
CREATE_BUILDING
CREATE_LEVEL
CREATE_SPACE
CREATE_ELEMENT
MOVE_ELEMENT
ROTATE_ELEMENT
RESIZE_ELEMENT
DELETE_ELEMENT
CHANGE_PARAMETER
ADD_CONSTRAINT
REMOVE_CONSTRAINT
MODIFY_REQUIREMENT
CREATE_VIEW
GENERATE_DRAWING
GENERATE_DOCUMENT_SET
RUN_VALIDATION
CREATE_REDLINE
RESOLVE_REDLINE
REGENERATE_AFFECTED_DOCUMENTS
```

The command vocabulary shall expand as the model matures.

---

24. ARIA COMMAND EXECUTION

Every command follows:

```
USER REQUEST
     ↓
ARIA INTERPRETATION
     ↓
STRUCTURED COMMAND
     ↓
COMMAND VALIDATION
     ↓
DEPENDENCY ANALYSIS
     ↓
MODEL TRANSACTION
     ↓
GEOMETRY UPDATE
     ↓
VALIDATION
     ↓
REVISION CREATION
     ↓
DOCUMENT IMPACT ANALYSIS
```

Aria does not directly write project records.

---

25. DESIGN PROPOSAL

Aria may also produce proposals rather than immediate commands.

Object

`DesignProposal`

Fields:

```
id
project_id
description
affected_objects
proposed_changes
rationale
constraints
estimated_impacts
status
```

Statuses:

```
PROPOSED
ACCEPTED
REJECTED
SUPERSEDED
```

This is particularly important for ambiguous or high-impact design changes.

---

26. VIEW

Object

`View`

Represents a model-derived view.

Fields:

```
id
project_id
view_type
level_id
section_plane
orientation
scale
visibility_settings
model_version_id
```

View types:

```
PLAN
ELEVATION
SECTION
DETAIL
REFLECTED_CEILING
ROOF_PLAN
FOUNDATION_PLAN
SITE_PLAN
AXONOMETRIC
PERSPECTIVE
DIAGRAM
```

---

27. DRAWING

Object

`Drawing`

Fields:

```
id
project_id
view_id
drawing_number
title
```

```
discipline
scale
status
model_version_id
revision_id
```

A Drawing is a documented representation of a View.

---

28. SHEET

Object

`Sheet`

Fields:

```
id
project_id
sheet_number
title
discipline
paper_size
template_id
drawing_ids
revision
status
```

Example:

```
A101 — Ground Floor Plan
A102 — Level 1 Floor Plan
A301 — Exterior Elevations
A401 — Building Sections
```

---

29. SHEET TEMPLATE

Object

`SheetTemplate`

Fields:

```
id
name
paper_size
orientation
title_block
graphic_standard
```

```
revision_block
version
```

The template controls presentation.

The Project Model controls content.

---

30. DIMENSION

Object

`Dimension`

Fields:

```
id
view_id
reference_objects
dimension_type
value
unit
tolerance
display_style
status
```

A dimension should preferably derive its value from geometry.

Manually overridden dimensions must be explicitly marked.

---

31. ANNOTATION

Object

`Annotation`

Fields:

```
id
view_id
annotation_type
text
target_object
position
style
status
```

Annotation types include:

```
ROOM_TAG
DOOR_TAG
WINDOW_TAG
KEYNOTE
GENERAL_NOTE
SECTION_REFERENCE
DETAIL_REFERENCE
ELEVATION_REFERENCE
GRID_REFERENCE
LEVEL_MARKER
```

---

32. SCHEDULE

Object

`Schedule`

Fields:

```
id
project_id
schedule_type
source_objects
columns
sorting
filtering
view_id
```

Examples:

```
DOOR_SCHEDULE
WINDOW_SCHEDULE
ROOM_SCHEDULE
FINISH_SCHEDULE
EQUIPMENT_SCHEDULE
```

Schedules are generated from model objects.

---

33. REDLINE

Object

`Redline`

Fields:

```
id
project_id
```

```
revision_id
drawing_id
target_object_id
markup_geometry
instruction
priority
created_by
created_at
status
resolution_id
```

Statuses:

```
OPEN
UNDER_REVIEW
IMPLEMENTED
VALIDATED
ACCEPTED
REJECTED
REQUIRES_CLARIFICATION
```

---

34. REDLINE RESOLUTION

Object

`RedlineResolution`

Fields:

```
id
redline_id
command_id
result
implemented_changes
validation_results
resolved_by
resolved_at
```

This connects the customer's markup directly to the model operation that resolved it.

---

35. CODE SOURCE

Object

`CodeSource`

Fields:

```
id
jurisdiction
authority
title
source_uri
version
effective_date
retrieved_at
content_hash
status
```

A source is never silently replaced.

A new version becomes a new source record.

---

36. CODE RULE

Object

`CodeRule`

Fields:

```
id
code_source_id
rule_identifier
citation
rule_type
requirement
applicability
threshold
unit
exceptions
verification_method
```

The rule is the machine-readable representation of a regulatory requirement.

---

37. CODE MANIFEST

Object

`CodeManifest`

Fields:

```
id
project_id
jurisdiction
municipality
```

```
province_state
country
source_ids
source_versions
effective_dates
retrieval_timestamp
content_hash
status
```

The manifest is immutable after project revision association.

A subsequent regulatory update produces a new manifest.

---

38. VALIDATION RESULT

Object

`ValidationResult`

Fields:

```
id
project_id
revision_id
rule_id
affected_objects
category
status
severity
description
source
created_at
```

Categories:

```
GEOMETRIC
SPATIAL
CUSTOMER_REQUIREMENT
ACCESSIBILITY
EGRESS
LIFE_SAFETY
CODE
ZONING
DOCUMENT
COORDINATION
```

Severity:

```
INFO
WARNING
ERROR
CRITICAL
REVIEW
```

---

39. VALIDATION RUN

Object

`ValidationRun`

Fields:

```
id
project_id
revision_id
model_version_id
code_manifest_id
started_at
completed_at
status
result_count
summary
```

A ValidationRun groups individual ValidationResults.

---

40. DOCUMENT PACKAGE

Object

`DocumentPackage`

Fields:

```
id
project_id
revision_id
model_version_id
code_manifest_id
drawing_ids
file_references
document_hash
generated_at
status
```

Statuses:

```
GENERATING
QA
READY_FOR_REVIEW
ACCEPTED
SUPERSEDED
ARCHIVED
```

41. DOCUMENT HASH

Every final generated artifact shall receive a content hash.

Conceptually:

```
HASH(file contents)
```

The hash is associated with:

```
Project
Revision
Model Version
Code Manifest
Generation Event
```

This establishes document provenance.

---

42. ACCEPTANCE RECORD

Object

`AcceptanceRecord`

Fields:

```
id
project_id
revision_id
model_version_id
document_package_id
code_manifest_id
customer_id
accepted_at
acceptance_status
acknowledgements
```

Statuses:

```
ACCEPTED
DECLINED
WITHDRAWN
SUPERSEDED
```

Acceptance does not automatically change the professional or municipal approval status of the project.

---

43. PROFESSIONAL REVIEW RECORD

Object

`ProfessionalReview`

Fields:

```
id
project_id
revision_id
reviewer_id
professional_role
review_scope
result
comments
reviewed_at
```

Possible results:

```
APPROVED
APPROVED_WITH_COMMENTS
REQUIRES_REVISION
REJECTED
```

This remains distinct from customer acceptance.

---

44. MUNICIPAL STATUS

Object

`ApprovalStatus`

Fields:

```
id
project_id
authority
application_reference
status
submitted_at
decision_at
notes
```

Statuses:

```
NOT_SUBMITTED
SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
```

```
WITHDRAWN
```

OneDraft records this state but does not presume municipal authority.

---

45. PROJECT EVENT

Object

`ProjectEvent`

Fields:

```
id
project_id
event_type
actor_id
revision_id
object_id
payload
timestamp
```

Events include:

```
PROJECT_CREATED
SPECIFICATION_UPDATED
MODEL_CREATED
COMMAND_SUBMITTED
COMMAND_EXECUTED
ELEMENT_CREATED
ELEMENT_MODIFIED
ELEMENT_DELETED
VALIDATION_COMPLETED
DRAWING_GENERATED
REDLINE_CREATED
REDLINE_RESOLVED
DOCUMENT_PACKAGE_CREATED
CUSTOMER_ACCEPTED
DOCUMENT_EXPORTED
```

The event record is append-only.

---

46. API — PROJECT

Create Project

`POST /api/v1/projects`

Retrieve Project
`GET /api/v1/projects/{project_id}`

Update Project
`PATCH /api/v1/projects/{project_id}`

Archive Project
`POST /api/v1/projects/{project_id}/archive`

Retrieve Project State
`GET /api/v1/projects/{project_id}/state`

---

47. API — REQUIREMENTS

Create Requirement
`POST /api/v1/projects/{project_id}/requirements`

List Requirements
`GET /api/v1/projects/{project_id}/requirements`

Modify Requirement
`PATCH /api/v1/requirements/{requirement_id}`

Verify Requirements
`POST /api/v1/projects/{project_id}/requirements/validate`

---

48. API — MODEL

Retrieve Model
`GET /api/v1/projects/{project_id}/model`

Retrieve Level
`GET /api/v1/levels/{level_id}`

Retrieve Space

`GET /api/v1/spaces/{space_id}`

Retrieve Element

`GET /api/v1/elements/{element_id}`

Create Element

`POST /api/v1/projects/{project_id}/elements`

Modify Element

`PATCH /api/v1/elements/{element_id}`

---

49. API — ARIA

Submit Natural-Language Request

`POST /api/v1/projects/{project_id}/aria/requests`

Submit Structured Command

`POST /api/v1/projects/{project_id}/commands`

Retrieve Command

`GET /api/v1/commands/{command_id}`

Approve Proposed Command

`POST /api/v1/commands/{command_id}/approve`

Reject Command

`POST /api/v1/commands/{command_id}/reject`

The AI request endpoint translates natural language into structured commands or proposals.

---

50. API — REVISIONS

Create Revision

`POST /api/v1/projects/{project_id}/revisions`

Retrieve Revision

`GET /api/v1/revisions/{revision_id}`

Compare Revisions

`GET /api/v1/projects/{project_id}/revisions/compare`

Restore Revision

`POST /api/v1/revisions/{revision_id}/restore`

Restoration creates a new revision based on the selected historical state.

It does not destroy history.

---

51. API — REDLINES

Create Redline

`POST /api/v1/projects/{project_id}/redlines`

List Redlines

`GET /api/v1/projects/{project_id}/redlines`

Resolve Redline

`POST /api/v1/redlines/{redline_id}/resolve`

Reject Redline

`POST /api/v1/redlines/{redline_id}/reject`

Request Clarification

`POST /api/v1/redlines/{redline_id}/clarification`

---

52. API — DRAWINGS

Generate View

`POST /api/v1/projects/{project_id}/views/generate`

Generate Drawing
`POST /api/v1/projects/{project_id}/drawings/generate`

Generate Sheet
`POST /api/v1/projects/{project_id}/sheets/generate`

Generate Document Set
`POST /api/v1/projects/{project_id}/documents/generate`

Retrieve Sheet
`GET /api/v1/sheets/{sheet_id}`

---

53. API — VALIDATION

Run Validation
`POST /api/v1/projects/{project_id}/validation`

Retrieve Validation Run
`GET /api/v1/validation-runs/{run_id}`

Retrieve Results
`GET /api/v1/validation-runs/{run_id}/results`

Validate Drawing Set
`POST /api/v1/projects/{project_id}/validation/drawings`

---

54. API — CODE

Resolve Jurisdiction
`POST /api/v1/projects/{project_id}/code/jurisdiction`

Create Manifest
`POST /api/v1/projects/{project_id}/code/manifests`

Retrieve Manifest

```
GET /api/v1/code/manifests/{manifest_id}
```

Retrieve Applicable Rules

```
GET /api/v1/projects/{project_id}/code/rules
```

---

55. API — EXPORT

Export PDF

```
POST /api/v1/projects/{project_id}/exports/pdf
```

Export DXF

```
POST /api/v1/projects/{project_id}/exports/dxf
```

Retrieve Export

```
GET /api/v1/exports/{export_id}
```

Future interfaces:

```
DWG
IFC
BIM
OTHER
```

---

56. API — ACCEPTANCE

Submit Acceptance

```
POST /api/v1/projects/{project_id}/acceptance
```

Retrieve Acceptance

```
GET /api/v1/projects/{project_id}/acceptance
```

Withdraw Acceptance

```
POST /api/v1/acceptance/{acceptance_id}/withdraw
```

---

57. API — EVENTS

Retrieve Project Events

`GET /api/v1/projects/{project_id}/events`

Retrieve Object History

`GET /api/v1/objects/{object_id}/history`

The event stream provides project provenance.

---

58. COMMAND TRANSACTION CONTRACT

Every mutating command must provide:

```
project_id
actor
operation
targets
parameters
expected_revision
```

The `expected_revision` prevents stale clients from unknowingly modifying an outdated project state.

If the project has changed since the client loaded it, the system returns a conflict requiring synchronization.

---

59. OPTIMISTIC CONCURRENCY

Example:

User A loads:

`Revision 14`

User B modifies project:

`Revision 15`

User A attempts modification against:

`Revision 14`

OneDraft detects that the project is no longer at Revision 14.

The operation is not blindly applied.

The system returns:

```
MODEL_STATE_CHANGED
```

Aria or the user can then reconcile the modification against Revision 15.

---

60. MODEL INTEGRITY RULE

No API may create a drawing object that contradicts the Project Model.

A drawing is always derived from:

```
Project
+
Model Version
+
View Definition
+
Drawing Rules
```

If a drawing needs to change because the model changed, the drawing is regenerated.

---

61. CODE INTEGRITY RULE

A project revision requiring regulatory analysis must reference a Code Manifest.

The system shall therefore prevent a revision from being represented as fully validated when no applicable regulatory baseline has been established.

Where a jurisdiction cannot be resolved, the project may continue as:

```
CODE_REVIEW_REQUIRED
```

but not silently as:

```
CODE_VALIDATED
```

---

62. REDLINE INTEGRITY RULE

A redline is not complete merely because the markup has been drawn.

It is complete only when:

```
Redline
→
Command
→
```

```
Model Change
→
Validation
→
Affected Drawings
→
Resolution
```

have been processed.

---

63. DOCUMENT INTEGRITY RULE

A final DocumentPackage must identify:

```
Project_ID
Revision_ID
Model_Version
Code_Manifest
Drawing_Set
Validation_Run
Document_Hashes
```

The document package therefore remains reproducible.

---

64. API RESPONSE STANDARD

API responses should use a consistent envelope.

Conceptually:

```
{
    data: {},
    metadata: {},
    errors: []
}
```

Errors should include:

```
code
message
field
object_id
resolution
```

AI-generated explanations may be presented separately from machine-readable errors.

---

65. ERROR TAXONOMY

Initial error codes:

```
PROJECT_NOT_FOUND
OBJECT_NOT_FOUND
INVALID_COMMAND
INVALID_GEOMETRY
CONSTRAINT_CONFLICT
MODEL_STATE_CHANGED
VALIDATION_FAILED
CODE_BASELINE_REQUIRED
DRAWING_GENERATION_FAILED
DOCUMENT_GENERATION_FAILED
REDLINE_INVALID
PERMISSION_DENIED
EXPORT_FAILED
```

Errors should be deterministic wherever possible.

---

66. INTERNAL EVENT FLOW

A typical model modification becomes:

```
USER
 ↓
ARIA REQUEST
 ↓
ARIA COMMAND
 ↓
COMMAND VALIDATOR
 ↓
PROJECT MODEL
 ↓
GEOMETRY ENGINE
 ↓
DEPENDENCY GRAPH
 ↓
VALIDATION ENGINE
 ↓
REVISION
 ↓
DRAWING IMPACT ANALYSIS
 ↓
DRAWING WORKER
 ↓
DOCUMENT QA
 ↓
CUSTOMER
```

This is the central OneDraft runtime loop.

---

67. FIRST IMPLEMENTATION OBJECTS

The initial implementation should prioritize:

```
Organization
User
Project
Requirement
Site
Building
Level
Space
Element
Constraint
Relationship
ModelVersion
Revision
AriaCommand
View
Drawing
Sheet
Redline
CodeManifest
ValidationResult
DocumentPackage
AcceptanceRecord
ProjectEvent
```

These objects constitute the minimum viable domain.

---

68. FIRST IMPLEMENTATION OPERATIONS

The first command vocabulary should be limited to operations that can be tested reliably:

```
CREATE_PROJECT
CREATE_BUILDING
CREATE_LEVEL
CREATE_SPACE
CREATE_WALL
CREATE_DOOR
CREATE_WINDOW
MOVE_ELEMENT
RESIZE_ELEMENT
DELETE_ELEMENT
ADD_CONSTRAINT
MODIFY_REQUIREMENT
GENERATE_PLAN
GENERATE_ELEVATION
GENERATE_SECTION
GENERATE_SHEET
RUN_VALIDATION
CREATE_REDLINE
```

```
RESOLVE_REDLINE
GENERATE_DOCUMENT_PACKAGE
ACCEPT_REVISION
```

Additional operations are added only after their model semantics are defined.

---

69. FIRST END-TO-END TEST

The canonical first test project shall be:

Two-storey residential building

with:

Ground Floor

Upper Floor

Three bedrooms

Kitchen

Living area

Bathrooms

Stair

Exterior envelope

Doors

Windows

The test shall execute:

```
CREATE PROJECT
      ↓
CREATE BUILDING
      ↓
CREATE LEVELS
      ↓
CREATE SPACES
      ↓
CREATE ELEMENTS
      ↓
GENERATE PLAN
      ↓
GENERATE ELEVATIONS
      ↓
GENERATE SECTION
      ↓
GENERATE ARCH D SHEETS
      ↓
CUSTOMER REDLINE
```

```
      ↓
ARIA COMMAND
      ↓
MODEL REVISION
      ↓
DRAWING REGENERATION
      ↓
VALIDATION
      ↓
FINAL DOCUMENT PACKAGE
```

If this sequence succeeds without manual database intervention, the OneDraft core architecture has been successfully demonstrated.

---

70. CANONICAL DATA RELATIONSHIP

The fundamental relationship is:

```
ORGANIZATION
    └── USER
          └── PROJECT
                ├── REQUIREMENTS
                ├── SITE
                ├── BUILDING
                │     ├── LEVEL
                │     │     ├── SPACE
                │     │     └── ELEMENT
                │     └── ELEMENT
                │
                ├── CONSTRAINTS
                ├── RELATIONSHIPS
                ├── CODE MANIFEST
                ├── MODEL VERSION
                ├── REVISION
                ├── REDLINES
                ├── VALIDATION
                ├── VIEWS
                ├── DRAWINGS
                ├── SHEETS
                ├── DOCUMENT PACKAGES
                ├── ACCEPTANCE
                └── EVENTS
```

Aria operates across this structure through controlled commands.

---

71. THE ONEDRAFT CONTRACT

The following rules are now considered foundational implementation constraints:

Rule 1: The Project Model is the single source of truth.

Rule 2: Aria does not directly modify persistent project data.

Rule 3: All AI modifications become structured commands.

Rule 4: Every meaningful modification creates a versioned revision.

Rule 5: Geometry is derived from semantic model objects.

Rule 6: Drawings are derived from model state.

Rule 7: Schedules are derived from model state.

Rule 8: Redlines become structured model changes.

Rule 9: Regulatory baselines are versioned.

Rule 10: Code manifests are associated with project revisions.

Rule 11: Validation results are persistent records.

Rule 12: Customer acceptance is distinct from professional certification and municipal approval.

Rule 13: Historical project states are never silently destroyed.

Rule 14: Final documents identify the model, revision and regulatory baseline from which they were generated.

Rule 15: Every major project event is auditable.

---

72. VERSION 0.1 COMPLETION

The OneDraft architecture has now progressed from:

CONCEPT

to:

PROJECT DATA MODEL

to:

SYSTEM ARCHITECTURE

to:

MVP TECHNICAL STACK

to:

DOMAIN/API CONTRACT

The next engineering layer is therefore implementation-specific.

The next specification should define the **OneDraft Database Schema v0.1 and OpenAPI contract**, translating the domain objects above into concrete PostgreSQL tables, indexes, foreign keys, JSON structures, API request/response schemas, authentication scopes, and machine-readable interfaces.

That will provide the first genuine **developer build specification** for OneDraft.